

Phase 2 Report

Group 8 Report

CMPT 276 – Introduction to Software Engineering

Cao Mark	xmc2@sfu.ca
Lu Phung	hnl8@sfu.ca
Meng Jiyang	jma289@sfu.ca
Mopewou Manuel	mam65@sfu.ca
Nguyen Nguyen	nnn4@sfu.ca

March 10, 2026

Simon Fraser University

Contents

1	Overall Approach	2
2	Management Process	2
2.1	First Milestone (March 2nd): Personal Coding	2
2.2	Second Milestone (March 10th): UI Implementation	3
3	Adjustments and Modifications	3
4	External Libraries	3
5	Challenges	4
5.1	Technical Issues	4
5.2	Human Issues	4
6	Current Status and Future Work	4

1. Overall Approach

The primary goal of Phase 2 was to transform the initial design developed in Phase 1 into a functional codebase with a clear and modular structure. Rather than implementing the entire game system at once, the team divided the project into several logical components. Each member was responsible for implementing a cluster of related classes so that development could proceed in parallel while preserving a coherent system architecture.

2. Management Process

Because the timeline of Phase 2 overlapped with the period of midterm examinations, the team decided to extend the duration of the first milestone in order to maintain both progress and code quality.

2.1 First Milestone (March 2nd): Personal Coding

Based on the feature set and class diagram established in Phase 1, the team assigned individual implementation tasks to each member. The allocation of responsibilities and expected targets for the first milestone is shown in Figure 1.

Members	Task	Expected Target
Phung Lu	Board + Cell + Position <enum> TitleType	- The board: - correctly loaded from a map identifier, with proper width, height, start position, and exit position - determine whether a position is valid and walkable, and must support querying and removing items as the game progresses - Each cell: accurately store its tile type, item, and occupant
Nhat Nguyen	Actor + Main Character + MovingEnemy <enum> Direction	- Actors: only move to valid and walkable positions on the board. - The main character: handle directional input, update its position, and maintain a bounded movement history. - Moving enemies: update based on game ticks, respect their speed factor, and determine their next position by following the main character's recorded path.
Manuel Antonio	Items + Punishment + Reward (both Regular & Bonus) + ScoreCount	- Each item: correctly update the game state when collected - Regular rewards: reduce the remaining collectible count, bonus rewards must expire after a fixed number of ticks, and punishments must apply penalties
Mark Cao	GameEngine + InputHandler <enum> Status	- The engine: - correctly initialize the game in preview mode, advance the game on each tick, and transition to running mode after the preview period - update the main character and enemies in the correct order, handle bonus expiration, and reliably detect win or lose conditions. When the game ends: produce a complete result object
Ji	LevelSection + GameState + Result <enum> loseReason	- GameState mainly serves as a container for the runtime state of the game. To keep its responsibility focused, score-related calculations are delegated to the ScoreCount class, which manages different types of scores. When the game ends, a Result object is created to capture the final outcome. - Level represents one playable level; LevelManager stores available levels; LevelFactory creates levels based on difficulty; LevelConfig encapsulates the concrete configuration parameters.

Figure 1: Task allocation and expected targets for the first milestone

In the plan, the `GameState` class is assigned to a single member. However, group discussion decide the content and structure of `GameState`.

2.2 Second Milestone (March 10th): UI Implementation

- Implementing: push personal tasks into main branch; checking and changing the elements to make sure that all methods are compatible and able to work together.
- UI Parts: searching for tutorials related to building the user interface and conducting initial experiments.

3. Adjustments and Modifications

Mark Cao: Mark identified that the `PREVIEW` stage in the `Status` enumeration alone was not sufficient to clearly represent the different stages of the game.

Solution: A new enumeration class called `Turn` was added. This class separates the concept of game status from the concept of whose turn is currently being executed. The new enum explicitly distinguishes between the player's turn and the enemy's turn.

Effect: This improved the logical clarity of the game flow. It ensures that the system correctly reflects the planned gameplay sequence described in Phase 1: the main character appears first, allowing the player a short preview period to examine the map before the moving enemies begin chasing the player. By separating turns in this way, the engine can enforce the correct update order between the player and enemy actors.

Ji: The UML only included `Level` and `LevelManager` at first. It's sufficient at a high level but not detailed enough for implementation. Since our game supports different difficulty levels with different board sizes, reward counts, time limits, and enemy counts, storing all parameters in `Level` would make the class too heavy.

Solution: We refined the design by adding `LevelFactory` and `LevelConfig`. `LevelFactory` centralizes difficulty-based level creation, while `LevelConfig` stores the concrete numeric parameters.

Effect: The level system became more modular, easier to maintain and more in accordance with the implementation needs.

4. External Libraries

The main external framework used during this phase was the Java's built-in Swing library, which was employed to prototype the graphical user interface. Swing was selected because it is included in the standard Java runtime environment and does not require additional dependencies or complex configuration. This made it suitable for rapid experimentation with rendering the game board and visually displaying elements such as the player, tiles, and map structure.

5. Challenges

5.1 Technical Issues

The most significant technical challenge encountered during this phase was the design of the user interface. None of the team members had prior experience with graphical user interface development in Java, so each person initially explored potential approaches independently and conducted small-scale experiments. As the Phase 2 deadline approached, the most workable implementation was placed in the `ui` package, although that version still contained unresolved issues. Even so, the prototype was sufficient to demonstrate that the board could be rendered and that the intended layout was feasible in practice.

Another technical challenge involved designing the behaviour of the moving enemies. These enemies needed to follow the path previously taken by the main character while still moving at a slower speed so that gameplay remained fair and manageable. Determining how to record, store, and update this path required careful coordination between actor logic and the game engine.

In addition, the team faced repeated design decisions regarding the overall screen layout, including how the board, player, and other elements should be displayed in a clear and functional way. Most of these problems were gradually addressed through experimentation, iteration, and debugging.

5.2 Human Issues

The team also experienced several coordination-related challenges. Because members had different schedules, it was often difficult to arrange meetings that everyone could attend. As a result, the quality and efficiency of synchronous discussions were sometimes limited.

Most project decisions were therefore made through ongoing discussions in the team's WhatsApp group chat. Although this method supported asynchronous communication, it also introduced difficulties in maintaining a consistent flow of information among all members. At times, this lack of direct communication slowed down both coordination and decision-making.

6. Current Status and Future Work

- Core architecture of the game has been successfully implemented
- An experimental user interface prototype has been developed to render the board and temporarily test basic player movement visually
- However, the UI and technical parts have not done: scoring mechanism, level selection, start menu, and difficulty mechanism (gradually increasing)